Composants, Props, État Local et Interactions Utilisateur

Objectifs

À l'issue de cette session, vous serez capables de :

1. Concevoir et structurer des composants fonctionnels modulaires et réutilisables.
2. Transmettre des données d'un composant parent vers un composant enfant via les `props`.
3. Gérer l'état interne d'un composant de manière réactive avec le Hook `useState`.
4. Appliquer la règle fondamentale d'immutabilité lors de la mise à jour de tableaux ou d'objets dans l'état.
5. Intercepter les événements utilisateur et faire communiquer un composant enfant avec son parent (remontée d'état).

---

1. Le Modèle de Composants dans React

1.1 Qu'est-ce qu'un Composant ?

Dans React, l'interface utilisateur est découpée en briques indépendantes, isolées et réutilisables appelées **composants**. Chaque composant encapsule sa propre logique d'affichage, ses styles éventuels et son état interne.

Historiquement (avant 2019 et React 16.8), les composants étaient écrits sous forme de classes JavaScript (`class MonComposant extends React.Component`). Aujourd'hui, le standard absolu de l'industrie repose sur les **Composants Fonctionnels (Functional Components)** combinés aux **Hooks**.

Un composant fonctionnel obéit à trois règles syntaxiques strictes :

1. **C'est une fonction JavaScript standard** (déclarée avec `function` ou sous forme de fonction fléchée).
2. **Son nom commence impérativement par une majuscule (PascalCase)**, par exemple `StationCard` ou `HeaderBar`. C'est ainsi que React distingue une balise HTML native en minuscule (`<div>`, `<span>`) d'un composant React personnalisé (`<StationCard />`).
3. **Il retourne une description d'interface (JSX)** ou `null` s'il ne doit rien afficher.

```jsx
// Définition d'un composant fonctionnel simple
export default function SystemHeader() {
  return (
    <header className="system-header">
      <h1>Centre de Supervision Météorologique</h1>
      <p>Unité de surveillance réseau en temps réel</p>
    </header>
  );
}
```

1.2 Découpage Atomique et Arborescence

Construire une application React consiste à assembler des composants sous forme d'arbre hiérarchique, du plus général au plus spécifique :

- **`App`** (Composant racine)
    - **`Header`** (Barre de navigation et statut global)
    - **`Dashboard`** (Conteneur principal)
        - **`FilterBar`** (Barre de recherche et filtres)
        - **`StationList`** (Grille de stations)
            - **`StationCard`** (Carte individuelle répétée pour chaque station)
                - **`StatusBadge`** (Indicateur visuel d'état)
    - **`Footer`** (Pied de page)

Quand faut-il créer un nouveau composant ?

- **Principe de responsabilité unique (Single Responsibility Principle) :** Un composant ne devrait idéalement faire qu'une seule chose. S'il grandit trop et gère plusieurs préoccupations distinctes, il doit être découpé en sous-composants.
- **Réutilisabilité :** Si un bloc visuel (bouton, badge, carte, champ de saisie) apparaît à plusieurs endroits de l'application, il mérite son propre composant.
- **Lisibilité :** Un fichier de composant qui dépasse 150 à 200 lignes devient difficile à maintenir.

---

2. Les Propriétés (`props`) : Paramétrer les Composants

2.1 Concept et Passage de Props

Si un composant est une fonction JavaScript, les **`props` (propriétés)** sont l'unique objet argument reçu par cette fonction. Elles permettent à un composant parent de configurer et de transmettre des données à un composant enfant.

Sur le plan syntaxique, on passe des `props` à un composant exactement comme on ajoute des attributs à une balise HTML, à une différence majeure près : on peut y passer **n'importe quel type de donnée JavaScript** (chaînes, nombres, booléens, tableaux, objets, ou même des fonctions) en utilisant les accolades `{}`.

```jsx
// Composant Parent : App.jsx
import StationCard from './StationCard';

export default function App() {
  return (
    <main>
      <StationCard
        name="Toulouse-Blagnac"
        altitude={152}
        isActive={true}
        sensors={['Température', 'Vent', 'Pression']}
      />
      <StationCard
        name="Pic du Midi"
        altitude={2877}
        isActive={false}
        sensors={['Température', 'Hygrométrie']}
      />
    </main>
  );
}
```

2.2 Réception, Déstructuration et Valeurs par Défaut

Le composant enfant reçoit un objet unique `props` regroupant toutes les clés transmises par son parent. En pratique professionnelle, on utilise systématiquement la **déstructuration**

d'objet ES6 directement dans la signature de la fonction, ce qui permet également de définir des valeurs par défaut.

```jsx
// Composant Enfant : StationCard.jsx
export default function StationCard({
  name,
  altitude = 0,          // Valeur par défaut si altitude n'est pas fournie
  isActive = true,       // Valeur par défaut
  sensors = []
}) {
  return (
    <article className="card">
      <h2>Station : {name}</h2>
      <p>Altitude : {altitude} m</p>
      <p>Statut : {isActive ? 'En service' : 'Hors ligne'}</p>
      <p>Capteurs installés : {sensors.length}</p>
    </article>
  );
}
```

2.3 Règle d'Or : Les Props sont en Lecture Seule (Immutabilité)

Un composant doit agir comme une **fonction pure** vis-à-vis de ses `props` : il lui est formellement interdit de modifier les `props` qu'il reçoit.

```jsx
// INTERDIT : Mutation directe d'une prop
export default function StationCard(props) {
  props.altitude = 500; // Erreur ! Les props sont immuables (read-only)
  return <div>{props.altitude}</div>;
}
```

Si une donnée doit évoluer dans le temps suite à une action de l'utilisateur ou une réponse réseau, elle ne relève plus des `props`, mais de l'**état (`state`)**.

2.4 La Prop Spéciale `children`

Lorsqu'on écrit du contenu entre la balise ouvrante et la balise fermante d'un composant personnalisé, React injecte automatiquement ce contenu dans une prop réservée nommée **`children`**. Cette technique est fondamentale pour créer des composants d'habillage génériques (boîtes modales, panneaux, cartes conteneurs).

```jsx
// Composant conteneur générique : Panel.jsx
export function Panel({ title, children }) {
  return (
    <section className="panel-box">
      <header className="panel-header">
        <h3>{title}</h3>
      </header>
      <div className="panel-body">
        {children}
      </div>
    </section>
  );
}

// Utilisation dans un composant parent :
<Panel title="Alerte Vent Violent">
  <p>Rafales enregistrées à plus de 110 km/h.</p>
  <button>Acquitter l'alerte</button>
</Panel>
```

---

3. L'État Local (`state`) et le Hook `useState`

3.1 Pourquoi une Variable JavaScript Classique ne Suffit Pas ?

Un piège fréquent lorsque l'on débute en React consiste à utiliser une simple variable `let` pour stocker une donnée évolutive :

```jsx
// CONTRE-EXEMPLE : Ne fonctionne pas dans React
export default function CounterBroken() {
  let count = 0;
```

```
  function handleIncrement() {
    count = count + 1;
    console.log('Nouvelle valeur en mémoire :', count);
  }

  return (
    <button onClick={handleIncrement}>
      Compteur : {count}
    </button>
  );
}
```

Pourquoi l'écran reste-t-il figé sur `Compteur : 0` alors que la console affiche bien `1`, `2`, `3` ?

1. **Pas de déclenchement de rendu :** Modifier une variable locale `let` n'avertit pas React que l'interface doit être recalculée.
2. **Perte de mémoire entre deux rendus :** Même si React ré-exécutait la fonction `CounterBroken()`, la ligne `let count = 0;` serait relue depuis le début et réinitialiserait systématiquement la variable à `0`.

3.2 Anatomie du Hook `useState`

Pour doter un composant d'une mémoire persistante entre les rendus et déclencher automatiquement la mise à jour de l'interface, on utilise le Hook **`useState`** fourni par React.

```
```jsx
import { useState } from 'react';

export default function AlertCounter() {
  // Déclaration d'une variable d'état :
  // 1. alertCount : la valeur actuelle de l'état pour ce cycle de
rendu
  // 2. setAlertCount : la fonction (setter) permettant de modifier
l'état et de demander un re-rendu
  // 3. 0 : la valeur initiale (utilisée uniquement lors du tout
premier rendu)
  const [alertCount, setAlertCount] = useState(0);

  return (
    <div>
      <p>Alertes actives : {alertCount}</p>
```
```

```
      <button onClick={() => setAlertCount(alertCount + 1)}>
        Ajouter une alerte
      </button>
      <button onClick={() => setAlertCount(0)}>
        Réinitialiser
      </button>
    </div>
  );
}
```

3.3 Règles Fondamentales des Hooks

Tous les Hooks React (dont le nom commence par `use`, comme `useState`, `useEffect`, `useMemo`) obéissent à deux règles strictes imposées par le fonctionnement interne de React :

1. **Appel au niveau supérieur uniquement (Top-Level Only) :** N'appelez jamais `useState` à l'intérieur d'une condition (`if`), d'une boucle (`for`) ou d'une fonction imbriquée. Les Hooks doivent toujours être déclarés tout en haut de votre composant fonctionnel, dans un ordre strictement identique à chaque rendu.
2. **Appel depuis des fonctions React uniquement :** N'appelez des Hooks que depuis des composants fonctionnels React ou depuis vos propres Hooks personnalisés.

3.4 Asynchronisme, Batching et Mises à Jour Fonctionnelles

La fonction de mise à jour (`setCount`) ne modifie pas immédiatement la variable `count` sur la ligne suivante. Elle **planifie** un nouveau cycle de rendu avec la nouvelle valeur.

De plus, React regroupe plusieurs mises à jour d'état successives en un seul re-rendu pour optimiser les performances (mécanisme appelé **Batching**).

```
```jsx
// Problème : si count vaut 0, cliquer sur ce bouton donnera 1 (et non
3) !
function handleTripleIncrementBroken() {
  setCount(count + 1); // 0 + 1
  setCount(count + 1); // 0 + 1 (count vaut toujours 0 dans cet
instantané de rendu)
  setCount(count + 1); // 0 + 1
}

// Solution : la mise à jour fonctionnelle (Functional Update)
```
```

```
// Lorsque le nouvel état dépend de l'état précédent, passez une
fonction fléchée (prev => ...)
function handleTripleIncrementValid() {
  setCount(prevCount => prevCount + 1); // 0 -> 1
  setCount(prevCount => prevCount + 1); // 1 -> 2
  setCount(prevCount => prevCount + 1); // 2 -> 3
}
```

Bonne pratique : Utilisez systématiquement la syntaxe fonctionnelle `setState(prev => ...)` dès que la nouvelle valeur est calculée à partir de l'ancienne valeur (incrémentation, bascule booléenne `!prev`, ajout dans un tableau).

---

4. Immutabilité des Objets et des Tableaux dans l'État

En JavaScript, les types primitifs (nombres, chaînes, booléens) sont immuables par nature. En revanche, les **objets** et les **tableaux** sont manipulés par référence mémoire.

Pour détecter qu'un état a changé et déclencher un re-rendu, React effectue une comparaison stricte des références mémoire (`Object.is(ancienEtat, nouvelEtat)`). Si vous modifiez directement le contenu d'un objet ou d'un tableau existant (mutation avec `.push()`, `.splice()` ou `obj.prop = ...`) avant de le repasser à `setState`, **l'adresse mémoire reste identique : React considère que rien n'a changé et ne met pas à jour l'écran**.

4.1 Mettre à Jour un Objet Immuablement

Pour modifier un objet dans le `state`, vous devez créer une **copie superficielle (shallow copy)** à l'aide de l'opérateur de décomposition (`...`, appelé *Spread Operator*) et écraser uniquement les propriétés souhaitées.

```jsx
const [station, setStation] = useState({
  id: 'ST-01',
  name: 'Marseille-Marignane',
  temperature: 22.5,
  status: 'online'
});

// MAUVAIS : Mutation directe de l'objet existant (aucun re-rendu
déclenché)
```

```
function updateTempBroken(newTemp) {
  station.temperature = newTemp;
  setStation(station);
}

// BON : Création d'un nouvel objet copiée avec la propriété mise à
jour
function updateTempValid(newTemp) {
  setStation(prevStation => ({
    ...prevStation,
    temperature: newTemp
  }));
}
```

4.2 Mettre à Jour un Tableau Immuablement

De la même manière, toutes les méthodes qui mutent un tableau sur place sont proscrites sur un tableau d'état.

Opération souhaitée	**Méthode mutatrice (INTERDITE sur le state)**	**Approche immuable recommandée (CRÉE un nouveau tableau)**
Ajouter un élément	`array.push()`, `array.unshift()`	`[...prevArray, newItem]` ou `concat()`
Supprimer un élément	`array.splice()`, `array.pop()`	`prevArray.filter(item => item.id !== targetId)`
Modifier un élément	`array[i] = newValue`	`prevArray.map(item => item.id === id ? { ...item, ... } : item)`
Trier ou inverser	`array.sort()`, `array.reverse()`	`[...prevArray].sort(...)` (copier avant de trier)

Exemples types de manipulation d'un tableau dans le state

```jsx
const [measures, setMeasures] = useState([
  { id: 1, value: 18.2 },
```

```
  { id: 2, value: 19.5 }
]);

// 1. Ajouter une mesure à la fin
const addMeasure = (newValue) => {
  const newEntry = { id: Date.now(), value: newValue };
  setMeasures(prev => [...prev, newEntry]);
};

// 2. Supprimer une mesure par son identifiant
const removeMeasure = (idToDelete) => {
  setMeasures(prev => prev.filter(item => item.id !== idToDelete));
};

// 3. Modifier la valeur d'une mesure existante
const updateMeasure = (idToUpdate, updatedValue) => {
  setMeasures(prev =>
    prev.map(item =>
      item.id === idToUpdate ? { ...item, value: updatedValue } : item
    )
  );
};
```

---

5. Gestion des Interactions et Communication Enfant -> Parent

5.1 Les Événements Synthétiques (Synthetic Events)

Dans React, on attache des écouteurs d'événements directement sur les éléments JSX avec une syntaxe **camelCase** (`onClick`, `onChange`, `onSubmit`, `onMouseEnter`).

React enveloppe les événements natifs du navigateur dans un objet standardisé appelé **`SyntheticEvent`**, garantissant un comportement strictement identique quel que soit le navigateur utilisé.

Piège classique : passer la référence de la fonction vs l'appeler

```jsx
// MAUVAIS : La fonction alert() s'exécute immédiatement pendant le rendu !
<button onClick={alert('Clic !')}>Cliquer</button>
```

```
// BON (Sans argument) : On passe la référence de la fonction
<button onClick={handleAction}>Cliquer</button>

// BON (Avec argument) : On l'encapsule dans une fonction fléchée anonyme
<button onClick={() => handleDelete(station.id)}>Supprimer</button>
```

5.2 Faire Remonter l'Information (Lifting State Up & Callbacks)

Puisque le flux de données est unidirectionnel (du parent vers l'enfant), comment un composant enfant (`StationCard`) peut-il demander à son parent (`App`) de modifier la liste des stations ou de sélectionner une station active ?

La solution suit un schéma en 3 étapes :

1. Le **composant Parent** possède l'état (`useState`) et définit une fonction de modification (par exemple `handleSelectStation`).
2. Le **Parent** transmet cette fonction en tant que `prop` au composant Enfant (par exemple `onSelect={handleSelectStation}`).
3. Lors d'un clic utilisateur, l'**Enfant** appelle la fonction reçue dans ses `props` en lui passant ses propres informations en paramètre (`onSelect(id)`).

```jsx
// 1. Composant Parent : Dashboard.jsx
import { useState } from 'react';
import StationItem from './StationItem';

export default function Dashboard() {
  const [selectedId, setSelectedId] = useState(null);

  const stations = [
    { id: 'TLS', name: 'Toulouse' },
    { id: 'BDX', name: 'Bordeaux' }
  ];

  return (
    <div>
      <p>Station sélectionnée : {selectedId || 'Aucune'}</p>
      {stations.map(st => (
        <StationItem
          key={st.id}
```

```
          id={st.id}
          name={st.name}
          onSelect={(id) => setSelectedId(id)}
        />
      ))}
    </div>
  );
}

// 2. Composant Enfant : StationItem.jsx
export default function StationItem({ id, name, onSelect }) {
  return (
    <div className="station-item">
      <span>{name} ({id})</span>
      <button onClick={() => onSelect(id)}>
        Sélectionner cette station
      </button>
    </div>
  );
}
```

5.3 État Dérivé : Ce qui ne doit PAS aller dans `useState`

Une règle essentielle d'architecture React consiste à **ne jamais dupliquer dans le `state` une valeur qui peut être calculée à la volée** à partir de `props` ou d'un `state` existant lors du rendu. C'est ce qu'on appelle l'**état dérivé (Derived State)**.

```jsx
export default function StationsSummary() {
  const [stations, setStations] = useState([
    { id: 1, name: 'Station A', active: true },
    { id: 2, name: 'Station B', active: false },
    { id: 3, name: 'Station C', active: true }
  ]);

  // INUTILE ET DANGEREUX : Ne créez pas un second useState pour le total actif !
  // Calculer directement la valeur dans le corps de la fonction :
  // Elle sera automatiquement recalculée à chaque re-rendu de StationsSummary.
  const totalStations = stations.length;
```

```
  const activeCount = stations.filter(s => s.active).length;

  return (
    <p>
      Stations opérationnelles : {activeCount} sur {totalStations}
    </p>
  );
}
```

---

6. Synthèse comparative : Props vs State

Caractéristique	**`props` (Propriétés)**	**`state` (État Local)**
Origine de la donnée	Transmise par le composant parent	Créée et gérée à l'intérieur du composant lui-même (`useState`)
Mutabilité	Immuable (Lecture seule pour le composant qui la reçoit)	Modifiable uniquement via sa fonction setter (`set...`)
Rôle architectural	Configurer un composant enfant et lui passer des callbacks	Mémoriser des données évolutives liées aux interactions
Déclenche un re-rendu ?	Oui, lorsque le parent se re-rend avec de nouvelles `props`	Oui, dès que la fonction setter est appelée avec une nouvelle référence